

Centro Universitario de Ciencias Exactas e
Ingenierías (CUCEI)

SEMINARIO DE SOLUCION DE PROBLEMAS
DE TRADUCTORES DE LENGUAJES II

Etapa del proyecto analizador léxico
completo.

Mateo Sebastián serrano Pérez

25/01/2026



1. Introducción

En el presente proyecto se desarrolla un analizador léxico completo, constituye la primera fase dentro del proceso de compilación y tiene como objetivo principal analizar el código fuente carácter por carácter para identificar patrones válidos del lenguaje.

Durante este análisis, el código fuente es transformado en una secuencia ordenada de tokens o símbolos léxicos, Y Cada token representa una unidad significativa del lenguaje, como identificadores, números, operadores o palabras reservadas. Estos tokens serán utilizados posteriormente por el analizador sintáctico para verificar la estructura del programa.

2. Objetivos

- Implementar un analizador léxico funcional utilizando el lenguaje Python.
- Reconocer correctamente todos los símbolos léxicos definidos en la especificación del proyecto.
- Aplicar un algoritmo basado en un bucle infinito y una estructura tipo switch.
- Manejar adecuadamente los errores léxicos que puedan presentarse durante el análisis.
- Reportar la línea y columna de cada token reconocido para facilitar la depuración.

3. Descripción General del Analizador

El analizador léxico fue implementado en Python debido a su claridad sintáctica y facilidad para el manejo de cadenas. La clase principal del sistema se encarga de recorrer el código fuente utilizando un bucle infinito (while True), lo que permite procesar el texto hasta llegar al final del archivo.

Para determinar el tipo de cada token, se utiliza una estructura de control basada en condiciones if/elif, la cual simula el comportamiento de una estructura switch. Dependiendo del carácter leído, el analizador decide si debe reconocer un identificador, número, cadena, operador o símbolo especial.

4. Símbolos Léxicos Reconocidos

El programa es capaz de reconocer los siguientes símbolos léxicos:

- Identificadores formados por letras y dígitos.
- Números enteros y números reales.
- Cadenas de texto delimitadas por comillas dobles.
- Palabras reservadas como if, while, return, else, int, float y void.
- Operadores aritméticos (+, -, *, /).
- Operadores relacionales y lógicos (>, <, >=, <=, ==, !=, &&, ||).
- Símbolos especiales como paréntesis, llaves, comas y punto y coma.

5. Manejo de Errores

El analizador léxico incluye un mecanismo de detección de errores con el objetivo de identificar entradas inválidas. Entre los errores que pueden detectarse se encuentran caracteres no reconocidos por el lenguaje, operadores incompletos como '&' o '|', y cadenas de texto que no cuentan con una comilla de cierre.

Cuando se detecta un error, el sistema genera un mensaje indicando el tipo de error, así como la línea y columna exacta donde ocurrió, lo cual facilita la corrección del código fuente.

6. Pruebas y Resultados

Para verificar el correcto funcionamiento del analizador léxico, se realizaron diversas pruebas utilizando fragmentos de código escritos en el lenguaje de prueba definido para el proyecto.

Las pruebas incluyeron declaraciones de variables, uso de estructuras de control como if y while, operaciones aritméticas y lógicas, así como el manejo de errores intencionales. En todos los casos, el analizador generó correctamente la lista de tokens correspondiente y reportó los errores cuando fue necesario.

Los resultados obtenidos demuestran que el analizador cumple con los requisitos establecidos y produce salidas consistentes.

```
=====
Código a analizar:
=====
int main() {
    int x = 10;
    float y = 3.14;
    if (x > 5 && y < 10.0) {
        return x + y;
    } else {
        while (x != 0) {
            x = x - 1;
        }
    }
    return 0;
}
=====
```

```
=====
TOKENS RECONOCIDOS:
=====
Tipo                Valor                Línea  Columna
-----
TIPO                int                1      1
IDENTIFICADOR       main              1      5
PARENTESIS_ABRE     (                 1      9
PARENTESIS_CIERRA   )                 1     10
LLAVE_ABRE          {                 1     12
TIPO                int                2      5
IDENTIFICADOR       x                 2      9
ASIGNACION          =                 2     11
ENTERO              10                2     13
PUNTO_COMA          ;                 2     15
TIPO                float              3      5
IDENTIFICADOR       y                 3     11
ASIGNACION          =                 3     13
REAL                3.14              3     15
PUNTO_COMA          ;                 3     19
IF                  if                 4      5
PARENTESIS_ABRE     (                 4      8
IDENTIFICADOR       x                 4      9
OP_RELAC            >                 4     11
ENTERO              5                 4     13
OP_AND              &&                 4     15
IDENTIFICADOR       y                 4     18
OP_RELAC            <                 4     20
REAL                10.0              4     22
PARENTESIS_CIERRA   )                 4     26
LLAVE_ABRE          {                 4     28
RETURN              return             5      9
IDENTIFICADOR       x                 5     16
OP_SUMA             +                 5     18
IDENTIFICADOR       y                 5     20
PUNTO_COMA          ;                 5     21
LLAVE_CIERRA        }                 6      5
ELSE                else               6      7
LLAVE_ABRE          {                 6     12
WHILE               while              7      9
PARENTESIS_ABRE     (                 7     15
IDENTIFICADOR       x                 7     16
OP_IGUALDAD         !=                7     18
ENTERO              0                 7     21
PARENTESIS_CIERRA   )                 7     22
LLAVE_ABRE          {                 7     24
IDENTIFICADOR       x                 8     13
ASIGNACION          =                 8     15
IDENTIFICADOR       x                 8     17
OP_SUMA             -                 8     19
ENTERO              1                 8     21
PUNTO_COMA          ;                 8     22
LLAVE_CIERRA        }                 9      9
LLAVE_CIERRA        }                10     5
RETURN              return             11     5
ENTERO              0                 11    12
PUNTO_COMA          ;                 11    13
LLAVE_CIERRA        }                12     1
EOF                 $                 12     2
=====
Total de tokens: 54
Presiona Enter para salir...
```

7. Conclusión

Con Lo que se hizo en este analizador léxico se cumplen los requisitos establecidos para el proyecto, y pudimos ver como el programa reconoce correctamente los símbolos léxicos definidos, maneja errores de manera adecuada y presenta una estructura clara y organizada.

Este analizador puede ser utilizado como base para la implementación de un analizador sintáctico en etapas posteriores del desarrollo del compilador.